DS	**Session**	Queue	**Question Information**	Level 1 Challenge 53

Question description

Ramesh is an DS expert training youngsters struggling in DS to make them better.

Ramesh usually gives interesting problems to the youngsters to make them love the DS.

One such day Sathya provided to the youngsters to solve the task such that, delete an element in a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

1. Create a main function for the program.
2. In this main function accept the size and data of the queue by the user.
3. After that enqueue these element in the queue using an enqueue() function
4. Now to dequeue, we will be using an function that will remove elements from the queue one by one.
5. In this dequeue function first we will check if there are elements present in the queue or not.
6. If there are elements present in the queue then we will move the front of the queue by one step each time ,that is every time when an element is removed from the list.

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
5 6 3 1 2
```

EXPECTED OUTPUT

```
Dequeuing elements:
6 3 1 2
3 1 2
1 2
2
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
Dequeuing elements:
8 6 4 9 5 3 5
6 4 9 5 3 5
4 9 5 3 5
9 5 3 5
5 3 5
3 5
5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int data)
```

Test Case 2

KEYWORD

```
for(i=front;i<=rear;i++)
```

Test Case 3

KEYWORD

```
enqueue(data);
```

Test Case 4

KEYWORD

```
void dequeue()
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	250	59

Code

solution.c

```c
#include <stdio.h>

int queue[100];
int front = 0, rear = -1;

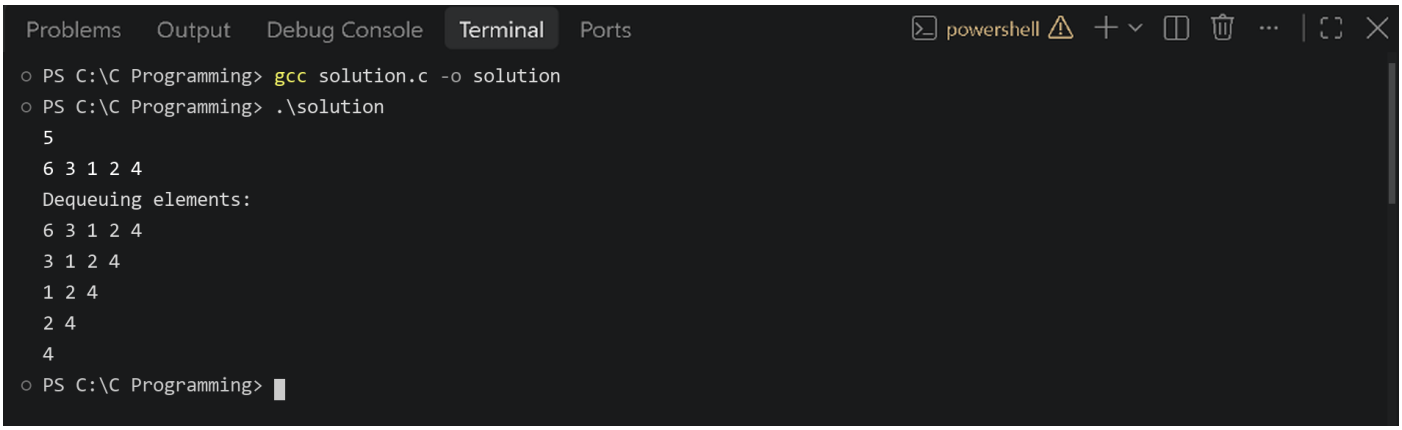
void enqueue(int data) {
    queue[++rear] = data;
}

void dequeue() {
    front++;
}

void display() {
    int i;
    for (i = front; i <= rear; i++) {
        printf("%d%s", queue[i], (i == rear) ? "" : " ");
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int data;
        scanf("%d", &data);
        enqueue(data);
    }
    printf("Dequeuing elements:\n");
    while (front <= rear) {
        display();
        dequeue();
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
6 3 1 2 4
Dequeuing elements:
6 3 1 2 4
3 1 2 4
1 2 4
2 4
4
PS C:\C Programming>